FECAP - Fundação de Comércio Álvares Penteado

Algoritmos e Lógica de Programação

Entrega 2 - Funções e Procedimentos aplicados ao jogo Factory Escape

Integrantes: André Ananias, Antonie Gabriel, João Vitor e Karine Vicente

Projeto: Factory Escape (jogo de plataforma 3D, Unity 6, linguagem C#)

Repositório: github.com/2026-2-MCC1/Projeto1

1. Objetivo

Este documento apresenta apenas a parte da aplicação dos algoritmos no projeto e os seus resultados. Os algoritmos abaixo usam **estruturas de funções e procedimentos**, além de decisão (if/else), repetição (for e foreach) e vetores. Todos os trechos foram escritos **sem atalhos e sem operador ternário**: cada decisão usa if/else completo. Os trechos são cópias do código-fonte do jogo, em `src/Entrega 1/Frontend/Assets/Scripts`.

2. Conceitos usados

Procedimento é uma sub-rotina que executa uma ação e não devolve valor (tipo `void`). **Função** é uma sub-rotina que calcula algo e devolve um valor com `return` (por exemplo `string`, `bool` ou `int`). Os dois podem receber parâmetros.

Algoritmo	Tipo	Parâmetros	Retorno	Estruturas
LevelName	Função	int l	string	decisão, vetor
GanharVida	Função	nenhum	bool	decisão composta
SyncLevelWithScene	Procedimento	string sceneName	nenhum	repetição (for), decisão, vetor
Victory	Procedimento	nenhum	nenhum	decisão if/else
NextLevel	Procedimento	nenhum	nenhum	chama outras sub-rotinas
SomarRocklets	Procedimento	int quantidade	nenhum	função Mathf.Max
Pintar	Procedimento	Color cor, float forca	nenhum	repetição (foreach), decisão
VoltarAoCheckpoint / SalvarCheckpoint	Procedimentos	Vector3 novoPonto	nenhum	atribuição de estado
Update (ObstaculoVaiEVem)	Procedimento	nenhum	nenhum	if / else if

3. Função LevelName (retorna o nome da fase)

Arquivo: `Telas/GameFlow.cs`

Recebe o número da fase e devolve o nome dela, lido do vetor `LevelNames`. Se o número estiver fora do intervalo, devolve texto vazio. Antes ela usava um operador ternário e foi reescrita com if completo.

```
public static string[] LevelNames =
{
    "DEPÓSITO DE DOCES",
    "FABRICA SUSPENSA",
    "CAMINHO PARA O PORTAL"
};

public static string LevelName(int l)
{
    // Número de fase válido: devolve o nome do vetor LevelNames
    if (l >= 1 && l <= LevelNames.Length)
    {
        return LevelNames[l - 1];
    }

    // Número inválido: devolve texto vazio
    return "";
}
```

Resultado no projeto: o nome da fase aparece na cutscene do portal (CutscenePortal). Teste de mesa:

Chamada	Condição	Valor retornado
LevelName(1)	1 >= 1 e 1 <= 3: verdadeiro	"DEPÓSITO DE DOCES"
LevelName(2)	verdadeiro	"FABRICA SUSPENSA"
LevelName(3)	verdadeiro	"CAMINHO PARA O PORTAL"
LevelName(0)	0 >= 1: falso	"" (vazio)
LevelName(4)	4 <= 3: falso	"" (vazio)

4. Procedimento SyncLevelWithScene (repetição sobre vetor)

Arquivo: `Telas/GameFlow.cs`

Percorre o vetor `LevelScenes` com um laço for, conta quantas posições têm o nome da cena atual e guarda o índice encontrado. Só atualiza a fase se achar exatamente uma correspondência.

```
public static void SyncLevelWithScene(string sceneName)
{
    int found = -1;
    int count = 0;

    for (int i = 0; i < LevelScenes.Length; i++)
    {
        if (LevelScenes[i] == sceneName)
        {
            found = i;
            count++;
        }
    }

    if (count == 1)
    {
        level = found + 1;
    }
}
```

Resultado no projeto: ao abrir a cena Fabrica_Parte2 diretamente, o laço encontra o índice 1 e a variável level passa a ser 2, mantendo a HUD e as metas corretas. Com uma cena que não é fase (por exemplo MainMenu), count fica 0 e level não muda.

5. Procedimentos Victory e NextLevel (decisão e encadeamento)

Arquivo: `Telas/GameFlow.cs`

Victory decide qual tela abrir ao concluir uma fase. NextLevel avança a fase sem passar do total, usando a função Mathf.Min, e chama o procedimento StartLevel.

```
public static void Victory()
{
    if (level >= TotalLevels)
    {
        // Última fase: tela de encerramento
        Load(EndScene);
    }
    else
    {
        // Fases 1 e 2: tela de Vitória
        Load(VictoryScene);
    }
}

public static void NextLevel()
{
    level = Mathf.Min(level + 1, TotalLevels);
    StartLevel();
}
```

Resultado no projeto: teste de mesa com TotalLevels = 3:

level antes	Victory() abre	NextLevel(): level depois
1	Vitoria	2 (carrega Fabrica_Parte2)
2	Vitoria	3 (carrega Fabrica_Parte3)
3	FimDeJogo	3 (Mathf.Min impede passar de 3)

6. Função GanharVida (retorna verdadeiro ou falso)

Arquivo: `Jogador/InventarioJogador.cs`

Tenta recuperar um casco (vida). Devolve `false` se a tartaruga já morreu (vidas <= 0) ou já está com as vidas cheias. Caso contrário soma uma vida, toca o som e o brilho, e devolve `true`.

```
public bool GanharVida()
{
    if (vidas <= 0 || vidas >= vidasIniciais)
    {
        return false;
    }
    vidas++;
    ArcorFactory.Sons.Tocar("vida", 0.8f, 0f);
    ArcorFactory.Efeitos.Brilhos(transform.position + Vector3.up,
        new Color(0.4f, 1f, 0.5f), 20);
    return true;
}
```

Resultado no projeto: teste de mesa com vidasIniciais = 3:

vidas	Condição (vidas <= 0 ou vidas >= 3)	Retorno	vidas depois
0	verdadeira (0 <= 0)	false	0
1	falsa	true	2
2	falsa	true	3
3	verdadeira (3 >= 3)	false	3

7. Procedimento SomarRocklets (com parâmetro)

Arquivo: `Jogador/InventarioJogador.cs`

Soma ao contador de Rocklets a quantidade recebida. A função Mathf.Max garante que nunca entre valor negativo.

```
public void SomarRocklets(int quantidade)
{
    rocklets += Mathf.Max(0, quantidade);
}
```

Resultado no projeto: SomarRocklets(5) com rocklets = 10 resulta em 15; SomarRocklets(-3) soma 0 e o contador continua em 10. O contador é mostrado na HUD e comparado com a meta da fase (50, 30 e 110 Rocklets).

8. Procedimento Pintar (repetição foreach e decisão)

Arquivo: `Jogador/InventarioJogador.cs`

Aplica a cor de dano em todas as partes (Renderers) da tartaruga. Se a força for zero ou menor, remove o efeito. Um laço foreach percorre o vetor de partes.

```
void Pintar(Color cor, float forca)
{
    if (partesTartaruga == null)
    {
        return;
    }

    cor.a = forca;
    foreach (Renderer parte in partesTartaruga)
    {
        if (forca <= 0f)
        {
            parte.SetPropertyBlock(null);
            continue;
```

```
        }

        parte.GetPropertyBlock(bloco);
        bloco.SetColor("_CorPiscar", cor);
        parte.SetPropertyBlock(bloco);
    }
}
```

Resultado no projeto: ao levar dano a tartaruga pisca em vermelho e depois volta à cor normal quando a invencibilidade termina (forca chega a 0).

9. Procedimentos de checkpoint e congelamento

Arquivo: `Jogador/RespawnJogador.cs e Jogador/InventarioJogador.cs`

SalvarCheckpoint guarda a posição recebida como parâmetro. VoltarAoCheckpoint leva o jogador de volta a ela. CongelarJogador e Congelar travam o jogador ao entrar no portal de saída: sem mover, sem dano e sem Game Over.

```
public void SalvarCheckpoint(Vector3 novoPonto)
{
    pontoAtual = novoPonto;
}

public void VoltarAoCheckpoint()
{
    // O CharacterController precisa ser desligado para mudar a posição
    controle.enabled = false;
    transform.position = pontoAtual;
    controle.enabled = true;
}

public static void CongelarJogador()
{
    InventarioJogador inv = FindFirstObjectByType<InventarioJogador>();
    if (inv != null)
    {
        inv.Congelar();
    }
}

public void Congelar()
{
    Congelado = true;
    CancelInvoke(nameof(IrParaGameOver));
    invencivel = 99999f;
    if (movimento != null)
    {
        movimento.enabled = false;
    }
}
```

Resultado no projeto: depois de passar pelo checkpoint da fase 2, cair no vazio leva a tartaruga de volta àquele ponto e não ao início. Ao entrar no último portal, Congelar liga o estado Congelado: o jogador não se move, não perde vida e a tela de Game Over não aparece.

10. Procedimento Update do obstáculo ObstaculoVaiEVem (if / else if)

Arquivo: `Obstaculos/ObstaculoVaiEVem.cs`

Executado a cada quadro. Soma o deslocamento e, ao chegar num dos limites, trava no limite e inverte o sentido (1 vai, -1 volta).

```
void Update()
{
    deslocamento = deslocamento + velocidade * sentido * Time.deltaTime;

    if (deslocamento > distancia)
```

```
    {
        deslocamento = distancia;
        sentido = -1;
    }
    else if (deslocamento < -distancia)
    {
        deslocamento = -distancia;
        sentido = 1;
    }

    transform.position = pontoInicial + direcao * deslocamento;
}
```

Resultado no projeto: com distancia = 2,8 e velocidade = 3, a plataforma anda de -2,8 a +2,8 em linha reta e volta, repetindo o movimento durante a fase.

11. Conclusão

O jogo usa funções (que devolvem valor, como LevelName e GanharVida) e procedimentos (que executam ações, como NextLevel, Victory e Congelar). Eles trabalham com decisão, repetição e vetores, em código escrito sem operador ternário nos trechos deste documento. Os resultados dos algoritmos aparecem no jogo: nomes das fases, avanço entre fases, vidas, Rocklets, checkpoint, congelamento no portal e obstáculos em movimento.